VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

Shashank Gowda L (1BM23CS311)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Aug-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled " Bio Inspired Systems (23CS5BSBIS)" carried out by **Shashank Gowda L (1BM23CS311),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

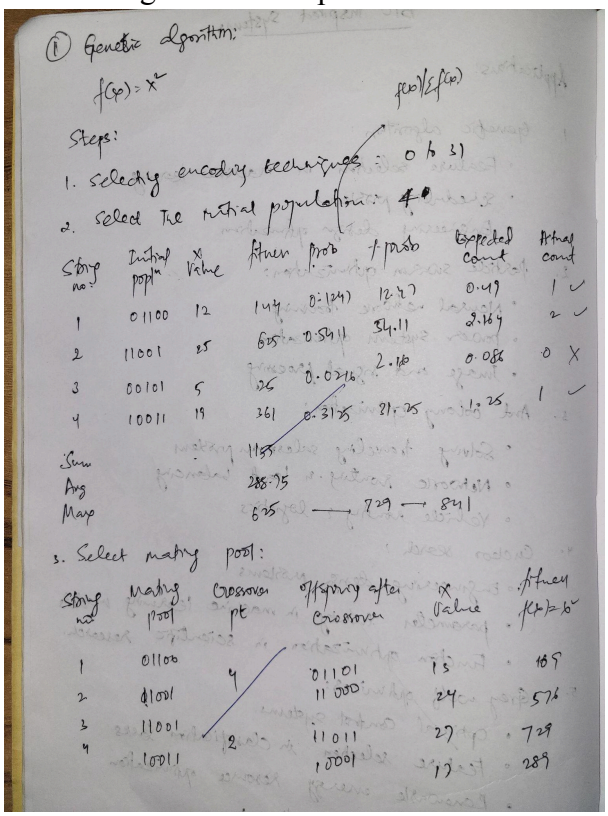
Rohith Vaidya K Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Github Link:
https://github.com/ShashankGowdaL69/BISLab

Program 1

Genetic Algorithm for Optimization Problems



① Genetic Algorithm:

$f(x) = x^2$

$f(x)/\sum f(x)$

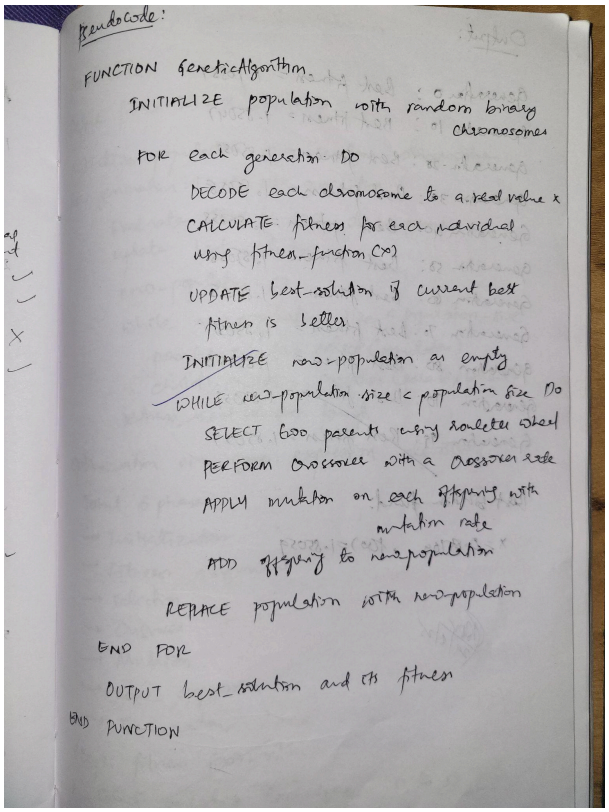
Steps:

1. Selecting encoding techniques : 0 & 1
2. Select the initial population : 4

String no	Initial popln	x value	fitness prob	f prob	Expected count	Actual count
1	01100	12	144	0.1241	0.49	1 ✓
2	11001	25	625	0.5411	2.164	2 ✓
3	00101	5	25	0.0216	0.086	0 X
4	10011	19	361	0.3125	1.25	1 ✓
Sum			1155			
Avg			288.75			
Max			625	→ 729 → 841		

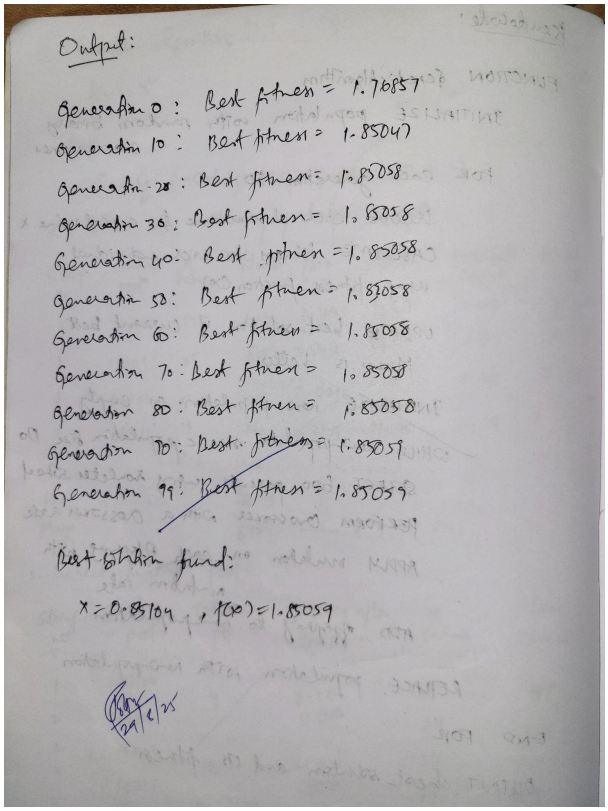
3. Select mating pool:

String no	Mating pool	Crossover pt	Offspring after Crossover	x value	fitness $f(x) = x^2$
1	01100	4	01101	13	169
2	11001		11000	24	576
3	11001	2	11011	27	729
4	10011		10001	17	289



Pseudocode:

```
FUNCTION GeneticAlgorithm
    INITIALIZE population with random binary chromosomes
    FOR each generation DO
        DECODE each chromosome to a real value x
        CALCULATE fitness for each individual using fitness_function(x)
        UPDATE best_solution if current best fitness is better
        INITIALIZE new_population as empty
        WHILE new_population size < population size DO
            SELECT two parents using roulette wheel
            PERFORM crossover with a crossover rate
            APPLY mutation on each offspring with mutation rate
            ADD offspring to new_population
        REPLACE population with new_population
    END FOR
    OUTPUT best_solution and its fitness
END FUNCTION
```



Output:

```
Generation 0 : Best fitness = 1.76857
Generation 10 : Best fitness = 1.85047
Generation 20 : Best fitness = 1.85058
Generation 30 : Best fitness = 1.85058
Generation 40 : Best fitness = 1.85058
Generation 50 : Best fitness = 1.85058
Generation 60 : Best fitness = 1.85058
Generation 70 : Best fitness = 1.85058
Generation 80 : Best fitness = 1.85058
Generation 90 : Best fitness = 1.85059
Generation 99 : Best fitness = 1.85059
```

Best solution found:

$x = 0.85104$, $f(x) = 1.85059$

29/4/25

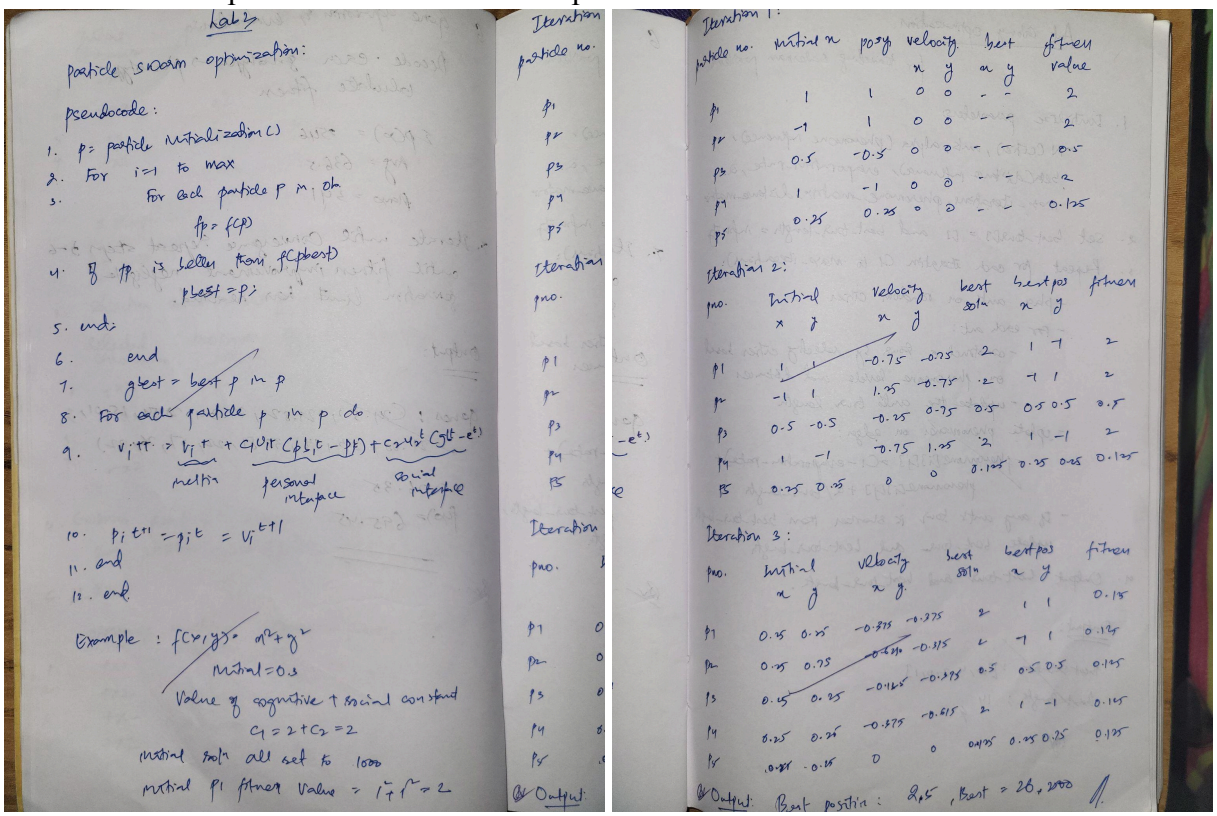
Code:

```
import random

def fitness_function(x):
    return x ** 2

def decode(chromosome):
    return int(chromosome, 2)

def evaluate_population(population):
    return [fitness_function(decode(individual)) for individual in population]

def select(population, fitnesses):
    total_fitness = sum(fitnesses)
    if total_fitness == 0:
        return random.choice(population)
    pick = random.uniform(0, total_fitness)
    current = 0
    for individual, fitness in zip(population, fitnesses):
        current += fitness
        if current > pick:
            return individual

def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, CHROMOSOME_LENGTH - 1)
        return (parent1[:point] + parent2[point:], parent2[:point] + parent1[point:])
    return parent1, parent2

def mutate(chromosome):
    new_chromosome = ''
    for bit in chromosome:
        if random.random() < MUTATION_RATE:
            new_chromosome += '0' if bit == '1' else '1'
        else:
            new_chromosome += bit
    return new_chromosome

def get_initial_population(size, length):
    population = []
    print(f"Enter {size} chromosomes (each of {length} bits, e.g., '10101'):")
    while len(population) < size:
        chrom = input(f"Chromosome {len(population)+1}: ").strip()
        if len(chrom) == length and all(bit in '01' for bit in chrom):
            population.append(chrom)
        else:
            print(f"Invalid input. Please enter a {length}-bit binary string.")
    return population
```

```
def genetic_algorithm():
    population = get_initial_population(POPULATION_SIZE, CHROMOSOME_LENGTH)
    best_solution = None
    best_fitness = float('-inf')

    for generation in range(GENERATIONS):
        fitnesses = evaluate_population(population)

        for i, individual in enumerate(population):
            if fitnesses[i] > best_fitness:
                best_fitness = fitnesses[i]
                best_solution = individual

        print(f"Generation {generation + 1}: Best Fitness = {best_fitness}, Best x =
{decode(best_solution)}")

        new_population = []
        while len(new_population) < POPULATION_SIZE:
            parent1 = select(population, fitnesses)
            parent2 = select(population, fitnesses)
            offspring1, offspring2 = crossover(parent1, parent2)
            offspring1 = mutate(offspring1)
            offspring2 = mutate(offspring2)
            new_population.extend([offspring1, offspring2])

        population = new_population[:POPULATION_SIZE]

    print("\nBest solution found:")
    print(f"Chromosome: {best_solution}")
    print(f"x = {decode(best_solution)}")
    print(f"f(x) = {fitness_function(decode(best_solution))}")

POPULATION_SIZE = 4
CHROMOSOME_LENGTH = 5
MUTATION_RATE = 0.01
CROSSOVER_RATE = 0.8
GENERATIONS = 20

if __name__ == "__main__":
    genetic_algorithm()
```

Program 2

Particle Swarm Optimization for Function Optimization



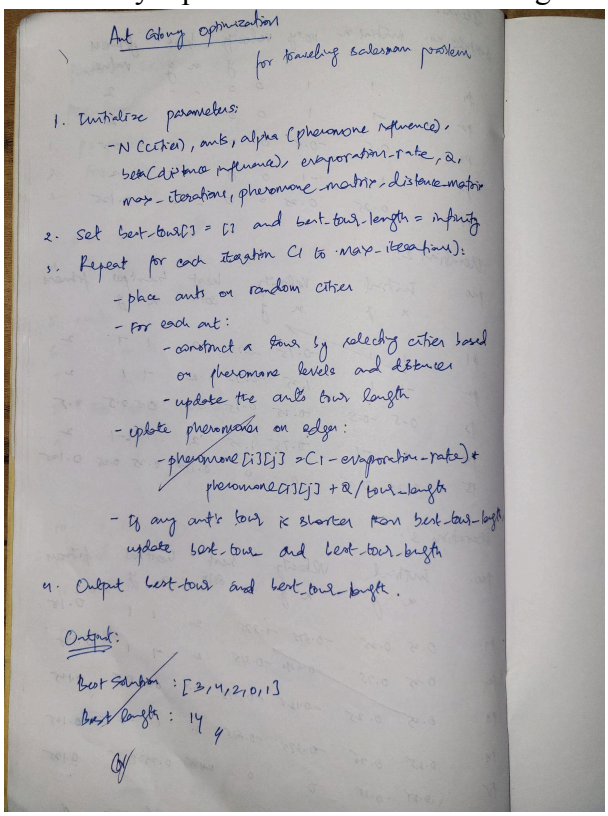
Code:

```
import random
import math

def fitness_function(x):
  return x**2

population_size = 100
num_genes = 8
mutation_rate = 0.01
crossover_rate = 0.8
num_generations = 5

population = [[random.randint(0, 1) for _ in range(num_genes)] for _ in range(population_size)]

def binary_to_decimal(binary_chromosome):
    decimal_value = 0
    for bit in binary_chromosome:
        decimal_value = decimal_value * 2 + bit
    return decimal_value

def evaluate_fitness(population):
    fitness_values = []
    for chromosome in population:
        x = binary_to_decimal(chromosome)
        fitness = fitness_function(x)
        fitness_values.append(fitness)
    return fitness_values

def select_parents(population, fitness_values):
    total_fitness = sum(fitness_values)
    if total_fitness == 0:
        return random.sample(population, 2)

    selection_probabilities = [f / total_fitness for f in fitness_values]
    parents_indices = random.choices(range(len(population)), weights=selection_probabilities, k=2)
    return [population[parents_indices[0]], population[parents_indices[1]]]

def crossover(parent1, parent2, crossover_rate):
    if random.random() < crossover_rate:
        point = random.randint(1, num_genes - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
        return child1, child2
    else:
        return parent1, parent2

def mutate(chromosome, mutation_rate):
    mutated_chromosome = chromosome[:]
```

```
    for i in range(num_genes):
        if random.random() < mutation_rate:
            mutated_chromosome[i] = 1 - mutated_chromosome[i]
    return mutated_chromosome

best_solution = None
best_fitness = -math.inf;

'''print("Starting Gene Expression Algorithm...")'''

for generation in range(num_generations):
    fitness_values = evaluate_fitness(population)

    current_best_fitness = max(fitness_values)
    current_best_index = fitness_values.index(current_best_fitness)
    current_best_individual_binary = population[current_best_index]
    current_best_individual_decimal = binary_to_decimal(current_best_individual_binary)

    if current_best_fitness > best_fitness:
        best_fitness = current_best_fitness
        best_solution = current_best_individual_binary
        best_individual_result = current_best_individual_decimal

    print(f"Generation {generation + 1}: Best Fitness = {current_best_fitness}, Best Individual
(decimal representation) = {current_best_individual_decimal}")

    new_population = []
    for _ in range(population_size // 2):
        parent1, parent2 = select_parents(population, fitness_values)
        child1, child2 = crossover(parent1, parent2, crossover_rate)
        new_population.append(mutate(child1, mutation_rate))
        new_population.append(mutate(child2, mutation_rate))

    population = new_population
```

Program 3

Ant Colony Optimization for the Traveling Salesman Problem



Ant Colony Optimization

for traveling salesman problem

1. Initialize parameters:
   - N (cities), ants, alpha (pheromone influence), beta (distance influence), evaporation_rate, Q, max_iterations, pheromone_matrix, distance_matrix
2. Set best_tour[] = [] and best_tour_length = infinity
3. Repeat for each iteration (1 to max_iterations):
   - place ants on random cities
   - For each ant:
     - construct a tour by selecting cities based on pheromone levels and distances
     - update the ants tour length
   - update pheromones on edges:
     - pheromone[i][j] = (1 - evaporation_rate) * pheromone[i][j] + Q / tour_length
   - If any ant's tour is shorter than best_tour_length update best_tour and best_tour_length
4. Output best_tour and best_tour_length.

Output:

Best Solution : [3, 4, 2, 0, 1]

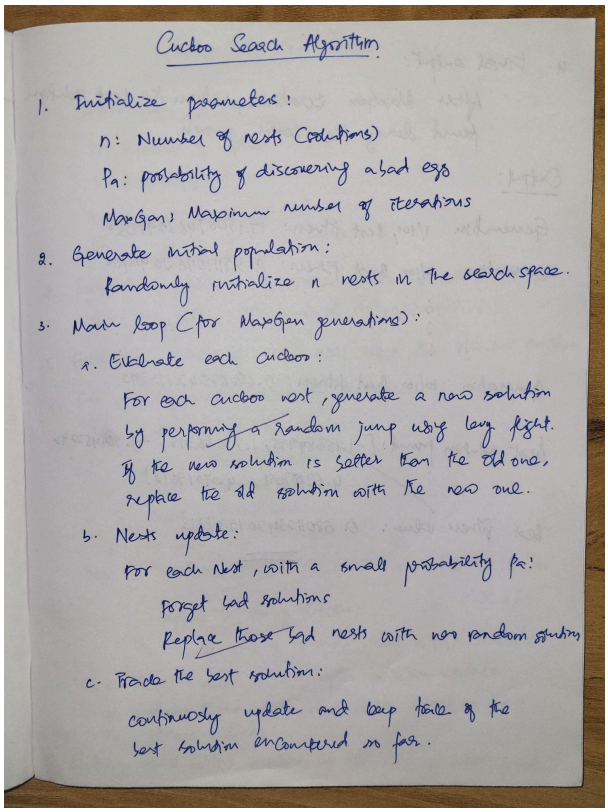
Best Length : 14

Code:

```
import numpy as np

def ant_colony_optimization(dist_matrix, n_ants, n_iterations, alpha=1, beta=2, rho=0.5, q=100):
    n = len(dist_matrix)
    pheromone = np.ones((n, n))
    visibility = 1 / (dist_matrix + np.eye(n) * 1e10)
    best_length = np.inf
    best_path = None

    for _ in range(n_iterations):
        all_paths = []
        all_lengths = []
        for _ in range(n_ants):
            path = [np.random.randint(n)]
            while len(path) < n:
                i = path[-1]
                prob = (pheromone[i] ** alpha) * (visibility[i] ** beta)
                prob[path] = 0
                prob /= prob.sum()
                next_city = np.random.choice(range(n), p=prob)
                path.append(next_city)
            length = sum(dist_matrix[path[i], path[i+1]] for i in range(n-1)) + dist_matrix[path[-1],
path[0]]
            all_paths.append(path)
            all_lengths.append(length)
            if length < best_length:
                best_length = length
                best_path = path
        pheromone *= (1 - rho)
        for path, length in zip(all_paths, all_lengths):
            for i in range(n-1):
                pheromone[path[i], path[i+1]] += q / length
            pheromone[path[-1], path[0]] += q / length
    return best_path, best_length

dist = np.array([
    [0, 2, 9, 10],
    [1, 0, 6, 4],
    [15, 7, 0, 8],
    [6, 3, 12, 0]
])

path, length = ant_colony_optimization(dist, n_ants=10, n_iterations=100)
print("Best path:", path)
print("Best length:", length)
```

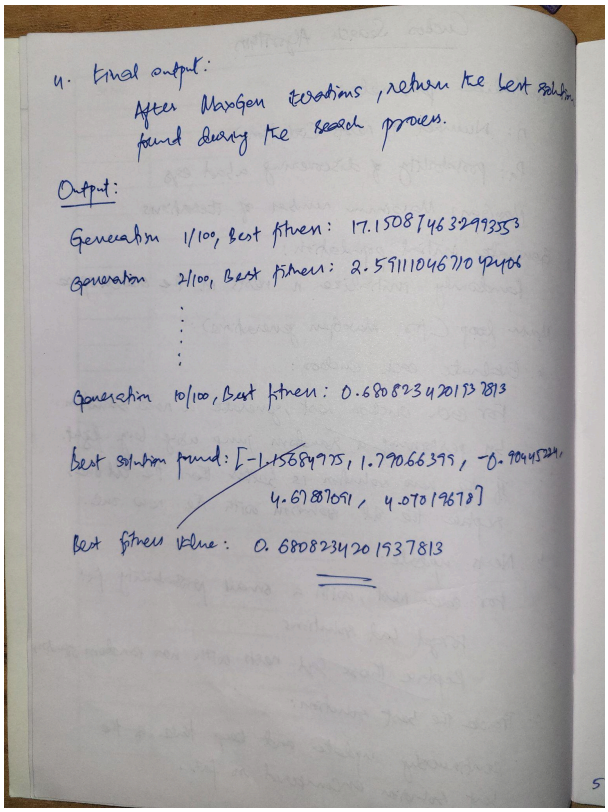
Program 4

Cuckoo Search



Cuckoo Search Algorithm

1. Initialize parameters:
   n: Number of nests (solutions)
   P_a: probability of discovering alien eggs
   MaxGen: Maximum number of iterations
2. Generate initial population:
   Randomly initialize n nests in the search space.
3. Main loop (for MaxGen generations):
   a. Evaluate each cuckoo:
      For each cuckoo nest, generate a new solution by performing a random jump using Levy flight.
      If the new solution is better than the old one, replace the old solution with the new one.
   b. Nests update:
      For each Nest, with a small probability P_a:
      forget bad solutions
      Replace those bad nests with new random solutions
   c. Track the best solution:
      continuously update and keep track of the best solution encountered so far.



4. Final output:
   After MaxGen iterations, return the best solution found during the search process.

Output:

Generation 1/100, Best fitness: 17.150874632993553

Generation 2/100, Best fitness: 2.5911104670104905

⋮

Generation 10/100, Best fitness: 0.6808234201537813

Best solution found: [-1.1568475, 1.7906639, -0.9044524, 4.6188709, 4.0701978]

Best fitness value: 0.6808234201537813

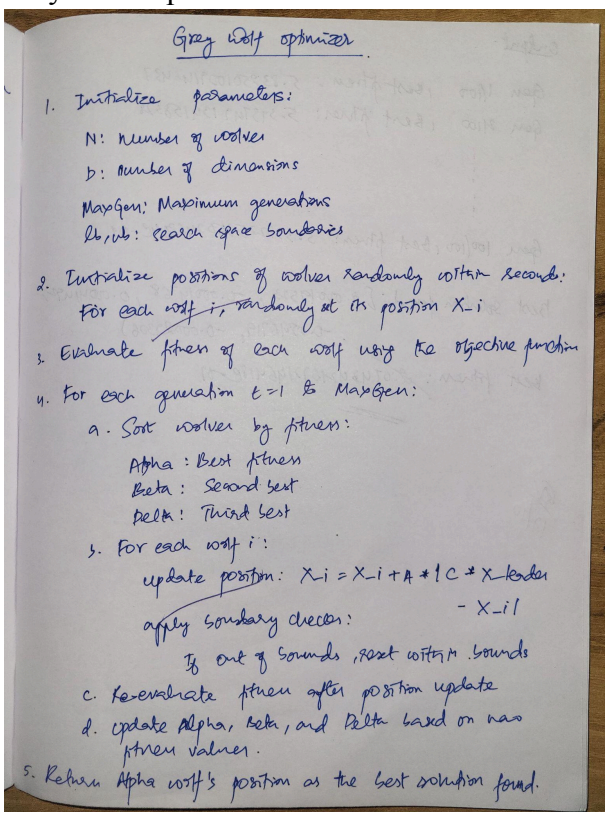
Code:

```
import numpy as np
from scipy.special import gamma  # import gamma from scipy.special

def levy_flight(dim, beta=1.5):
    sigma_u = (gamma(1 + beta) * np.sin(np.pi * beta / 2) /
               gamma((1 + beta) / 2) * beta * np.power(2, (beta - 1) / 2)) ** (1 / beta)
    u = np.random.normal(0, sigma_u, dim)
    v = np.random.normal(0, 1, dim)
    step = u / np.power(np.abs(v), 1 / beta)
    return step

def cuckoo_search(objective_function, n, dim, Pa=0.25, MaxGen=100, bounds=(-5, 5)):
    nests = np.random.uniform(bounds[0], bounds[1], (n, dim))
    fitness = np.apply_along_axis(objective_function, 1, nests)
    best_solution = nests[np.argmin(fitness)]
    best_fitness = np.min(fitness)

    for gen in range(MaxGen):
        new_nests = np.copy(nests)
        for i in range(n):
            step = levy_flight(dim)
            new_nests[i] = nests[i] + step
            new_nests[i] = np.clip(new_nests[i], bounds[0], bounds[1])
        new_fitness = np.apply_along_axis(objective_function, 1, new_nests)
        better_nests = new_fitness < fitness
        nests[better_nests] = new_nests[better_nests]
        fitness[better_nests] = new_fitness[better_nests]
        forget_idx = np.random.rand(n) < Pa
        nests[forget_idx] = np.random.uniform(bounds[0], bounds[1], (np.sum(forget_idx), dim))
        fitness = np.apply_along_axis(objective_function, 1, nests)
        current_best_fitness = np.min(fitness)
        if current_best_fitness < best_fitness:
            best_fitness = current_best_fitness
            best_solution = nests[np.argmin(fitness)]
        print(f"Generation {gen + 1}/{MaxGen}, Best Fitness: {best_fitness}")
    return best_solution, best_fitness

def sphere_function(x):
    return np.sum(x**2)

n = 30
dim = 5
MaxGen = 100
Pa = 0.25

best_solution, best_fitness = cuckoo_search(sphere_function, n, dim, Pa, MaxGen)

print("\nBest Solution Found: ", best_solution)
```

```
print("Best Fitness Value: ", best_fitness)
```

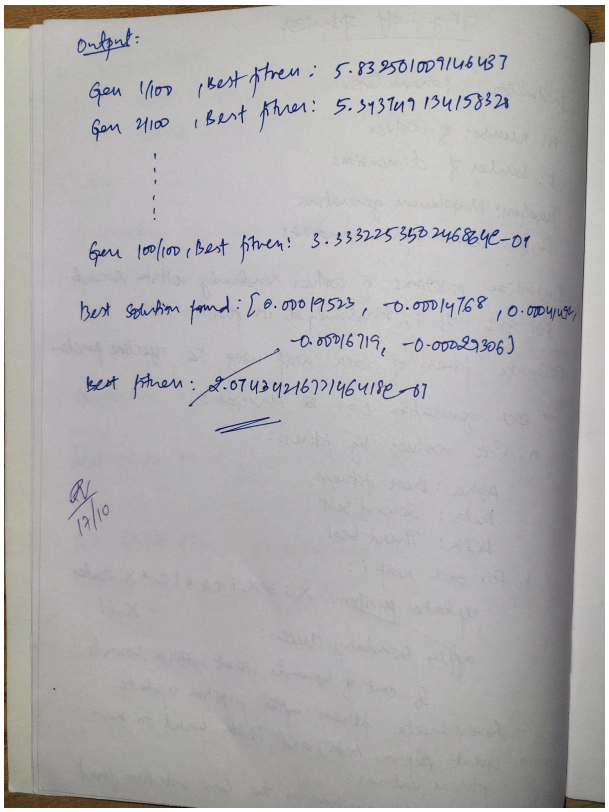
Program 5

Grey Wolf Optimizer



Grey wolf optimizer

1. Initialize parameters:
   - N: number of wolves
   - D: number of dimensions
   - MaxGen: Maximum generations
   - lb, ub: search space boundaries
2. Initialize positions of wolves randomly within bounds:
   For each wolf i, randomly set its position X_i
3. Evaluate fitness of each wolf using the objective function
4. For each generation t = 1 to MaxGen:
   a. Sort wolves by fitness:
      - Alpha: Best fitness
      - Beta: Second best
      - Delta: Third best
   b. For each wolf i:
      update position: X_i = X_i + A * |C * X_leader - X_i|
      apply boundary checks:
      If out of bounds, reset within bounds
   c. Re-evaluate fitness after position update
   d. Update Alpha, Beta, and Delta based on new fitness values.
5. Return Alpha wolf's position as the best solution found.



Output:

Gen 1/100, Best fitness: 5.832501009146437

Gen 2/100, Best fitness: 5.343749134153328

⋮

Gen 100/100, Best fitness: 3.333225353246884e-01

Best solution found: [0.00019523, -0.00014768, 0.00041458, -0.00016719, -0.00029306]

Best fitness: 2.074342167714641e-01

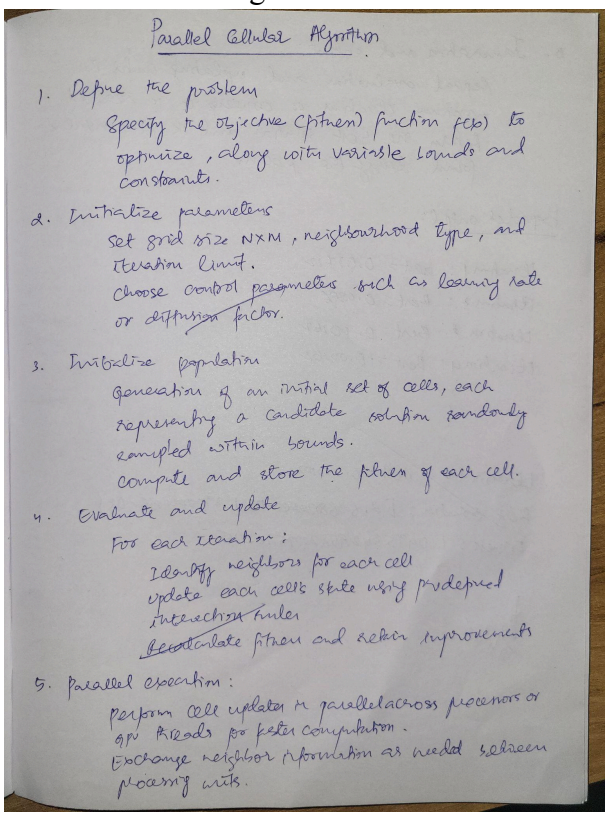
17/10

Code:

```
import numpy as np

def gwo(objective_function, n, dim, max_gen, lb, ub):
    wolves = np.random.uniform(lb, ub, (n, dim))
    fitness = np.apply_along_axis(objective_function, 1, wolves)
    alpha, beta, delta = np.copy(wolves), np.copy(wolves), np.copy(wolves)
    alpha_f, beta_f, delta_f = np.copy(fitness), np.copy(fitness), np.copy(fitness)

    for gen in range(max_gen):
        sorted_idx = np.argsort(fitness)
        alpha, beta, delta = wolves[sorted_idx[0]], wolves[sorted_idx[1]], wolves[sorted_idx[2]]
        alpha_f, beta_f, delta_f = fitness[sorted_idx[0]], fitness[sorted_idx[1]], fitness[sorted_idx[2]]

        A = 2 * np.random.rand(n, dim) - 1
        C = 2 * np.random.rand(n, dim)
        for i in range(n):
            D_alpha = np.abs(C[i] * alpha - wolves[i])
            D_beta = np.abs(C[i] * beta - wolves[i])
            D_delta = np.abs(C[i] * delta - wolves[i])

            wolves[i] = wolves[i] + A[i] * (D_alpha + D_beta + D_delta) / 3
            wolves[i] = np.clip(wolves[i], lb, ub)

        fitness = np.apply_along_axis(objective_function, 1, wolves)
        print(f"Gen {gen+1}/{max_gen}, Best Fitness: {min(fitness)}")

    return alpha, alpha_f

def sphere_function(x):
    return np.sum(x**2)

n = 30
dim = 5
max_gen = 50
lb, ub = -5, 5

best_solution, best_fitness = gwo(sphere_function, n, dim, max_gen, lb, ub)
print("\nBest Solution Found: ", best_solution)
print("Best Fitness: ", best_fitness)
```

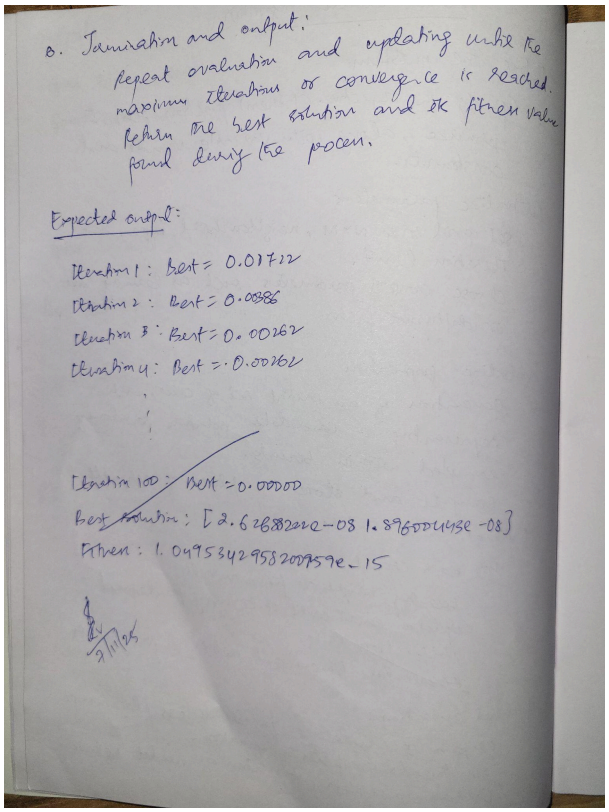
Program 6

Parallel Cellular Algorithm



Parallel Cellular Algorithm

1. Define the problem
   Specify the objective (fitness) function f(x) to optimize, along with variable bounds and constraints.
2. Initialize parameters
   Set grid size N×M, neighbourhood type, and iteration limit.
   Choose control parameters such as learning rate or diffusion factor.
3. Initialize population
   Generation of an initial set of cells, each representing a candidate solution randomly sampled within bounds.
   Compute and store the fitness of each cell.
4. Evaluate and update
   For each iteration:
   Identify neighbors for each cell
   Update each cell's state using predefined interaction rules
   Recalculate fitness and detect improvements
5. Parallel execution:
   Perform cell updates in parallel across processors or gpu threads for faster computation.
   Exchange neighbor information as needed between processing units.



6. Termination and output:
   Repeat evaluation and updating until the maximum iteration or convergence is reached.
   Return the best solution and its fitness value found during the process.

Expected output:

Iteration 1: Best = 0.01712
Iteration 2: Best = 0.0086
Iteration 3: Best = 0.00262
Iteration 4: Best = 0.00262
:
Iteration 100: Best = 0.00000
best solution: [2.62882e-08 1.89600e-08]
fitness: 1.04953429582097e-15

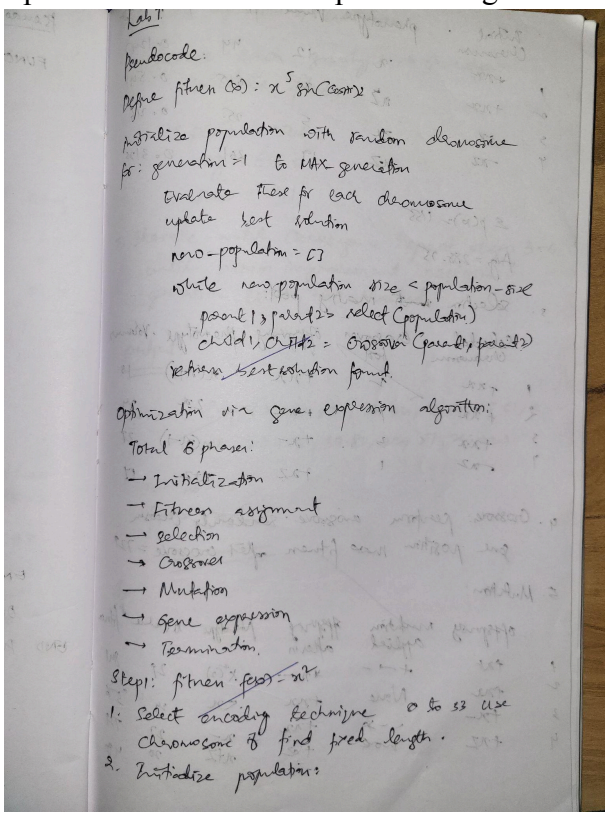
Code:

```
import random

# Parameters
POP_SIZE = 20
GENOME_LENGTH = 12
GENERATIONS = 25
MUTATION_RATE = 0.15

# Fitness function: maximize number of 1's
def fitness(individual):
    return sum(individual)

# Initialize population randomly
population = [[random.randint(0, 1) for _ in range(GENOME_LENGTH)] for _ in
range(POP_SIZE)]

# Tournament selection from neighbors
def tournament_selection(pop, fit_vals, idx, neighbors):
    best = idx
    for n in neighbors:
        if fit_vals[n] > fit_vals[best]:
            best = n
    return pop[best][:]

# Single-point crossover
def crossover(p1, p2):
    point = random.randint(1, GENOME_LENGTH - 1)
    return p1[:point] + p2[point:]

# Mutation
def mutate(individual):
    for i in range(GENOME_LENGTH):
        if random.random() < MUTATION_RATE:
            individual[i] = 1 - individual[i]

# 1D ring topology
def get_neighbors(i):
    left = (i - 1) % POP_SIZE
    right = (i + 1) % POP_SIZE
    return [left, right]

# Evolution process
for gen in range(GENERATIONS):
    fitness_values = [fitness(ind) for ind in population]
    new_population = []

    for i in range(POP_SIZE):
        neighbors = get_neighbors(i)
```

```
        p1 = tournament_selection(population, fitness_values, i, neighbors)
        p2 = tournament_selection(population, fitness_values, i, neighbors)
        child = crossover(p1, p2)
        mutate(child)
        new_population.append(child)

    population = new_population

    best_fit = max(fitness_values)
    avg_fit = sum(fitness_values) / len(fitness_values)
    print(f"Generation {gen + 1:02d}: Best = {best_fit}, Avg = {avg_fit:.2f}")

# Final results
final_fitness = [fitness(ind) for ind in population]
best_index = final_fitness.index(max(final_fitness))
best_solution = population[best_index]

print("\nBest Solution:", best_solution)
print("Best Fitness:", max(final_fitness))
```

Program 7

Optimization via Gene Expression Algorithm



Lab 7

Pseudocode:

Define fitness (x): x^5 sin(cos(x))

Initialize population with random chromosome

for generation = 1 to MAX generation

Evaluate fitness for each chromosome

update best solution

new_population = []

while new population size < population_size

parent1, parent2 = select(population)

child1, child2 = crossover(parent1, parent2)

return best solution found

Optimization via gene expression algorithm:

Total 6 phases:

→ Initialization

→ Fitness assignment

→ selection

→ Crossover

→ Mutation

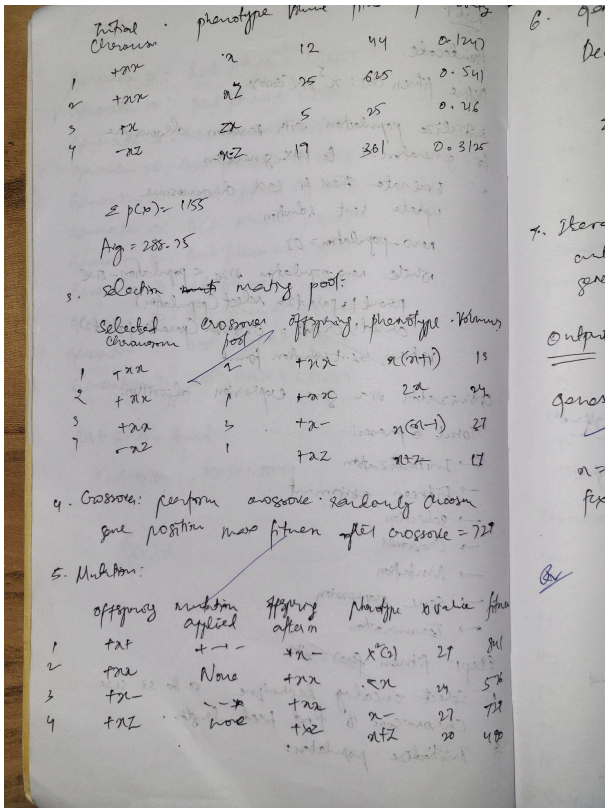
→ Gene expression

→ Termination.

Step1: fitness func = x^2

1. Select encoding technique 0 to 63 use Chromosome & find fixed length.

2. Initialize population:



	Initial Chromosome	phenotype			
1	+nx	x	12	44	0.1241
2	+nx	x2	25	625	0.541
3	+x	2x	5	25	0.46
4	-x2	x-2	19	361	0.3125

$\Sigma p(x) = 1155$

Avg = 288.75

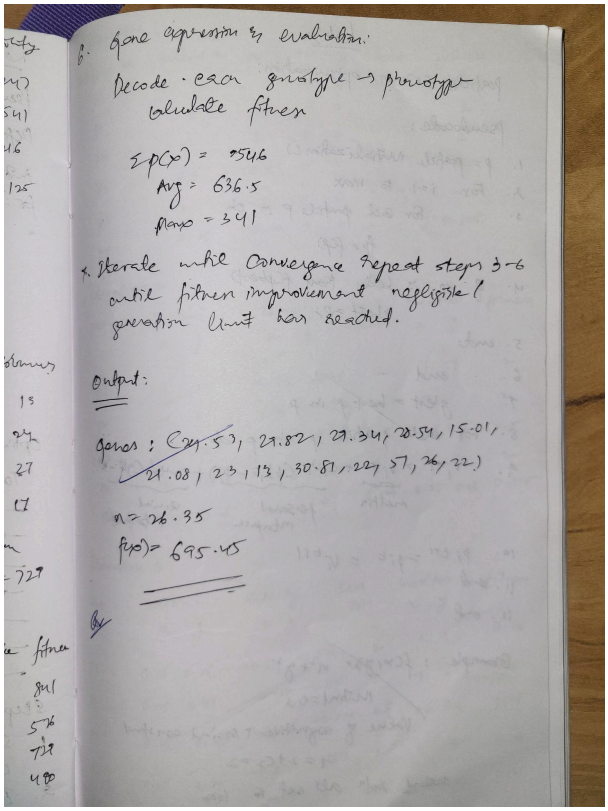
3. Selection mating pool:

	Selected Chromosome	crossover point	offspring	phenotype	fitness
1	+nx	2	+nx	x(x+n)	15
2	+nx	1	+nx	2x	24
3	+nx	3	+x-	x(x-1)	27
4	-x2	1	+22	x+2	17

4. Crossover: perform crossover randomly chosen gene position max fitness after crossover = 729

5. Mutation:

	offspring	mutation applied	offspring after m	Phenotype	fitness
1	+xf	+→-	*x-	$x^*(3)$	27
2	+xe	None	+nx	.5x	24
3	+x-	-→*	+x2	x-	27
4	+x2	None	+x2	x+2	20



6. Gene expression & evaluation:

Decode each genotype → phenotype

Calculate fitness

$\Sigma p(x) = 0546$

Avg = 636.5

max = 341

7. Iterate until convergence repeat steps 3-6 until fitness improvement negligible / generation limit has reached.

Output:

genes: (24.53, 27.82, 27.34, 28.54, 15.01, 21.08, 23, 13, 30.81, 22, 57, 26, 22)

n = 26.35

f(x) = 695.45

Code:

```
import random
import math

def fitness_function(x):
    return x * math.sin(10 * math.pi * x) + 2

POPULATION_SIZE = 6
GENE_LENGTH = 10
MUTATION_RATE = 0.05
CROSSOVER_RATE = 0.8
GENERATIONS = 20
DOMAIN = (-1, 2)

def random_gene():
    return random.uniform(DOMAIN[0], DOMAIN[1])

def create_chromosome():
    return [random_gene() for _ in range(GENE_LENGTH)]

def initialize_population(size):
    return [create_chromosome() for _ in range(size)]

def evaluate_population(population):
    return [fitness_function(express_gene(chrom)) for chrom in population]

def express_gene(chromosome):
    return sum(chromosome) / len(chromosome)

def select(population, fitnesses):
    total_fitness = sum(fitnesses)
    pick = random.uniform(0, total_fitness)
    current = 0
    for individual, fitness in zip(population, fitnesses):
        current += fitness
        if current > pick:
            return individual
    return random.choice(population)

def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, GENE_LENGTH - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
        return child1, child2
    return parent1[:], parent2[:]

def mutate(chromosome):
    new_chromosome = []
```

```
    for gene in chromosome:
        if random.random() < MUTATION_RATE:
            new_chromosome.append(random_gene())
        else:
            new_chromosome.append(gene)
    return new_chromosome

def gene_expression_algorithm():
    population = initialize_population(POPULATION_SIZE)
    best_solution = None
    best_fitness = float("-inf")

    for generation in range(GENERATIONS):
        fitnesses = evaluate_population(population)

        for i, chrom in enumerate(population):
            if fitnesses[i] > best_fitness:
                best_fitness = fitnesses[i]
                best_solution = chrom[:]

        print(f"Generation {generation+1}: Best Fitness = {best_fitness:.4f}, Best x =
{express_gene(best_solution):.4f}")

        new_population = []
        while len(new_population) < POPULATION_SIZE:
            parent1 = select(population, fitnesses)
            parent2 = select(population, fitnesses)
            offspring1, offspring2 = crossover(parent1, parent2)
            offspring1 = mutate(offspring1)
            offspring2 = mutate(offspring2)
            new_population.extend([offspring1, offspring2])

        population = new_population[:POPULATION_SIZE]

    print("\nBest solution found:")
    print(f"Genes: {best_solution}")
    x_value = express_gene(best_solution)
    print(f"x = {x_value:.4f}")
    print(f"f(x) = {fitness_function(x_value):.4f}")

if __name__ == "__main__":
    gene_expression_algorithm()
```